Version 2.0

January 2026

OBJECT-ORIENTED PROGRAMMING

MODULE 1 - BASIC JAVA

Author:

Ir. Galih Wasis Wicaksono, S.Kom., M.Cs.

Yossua Agung Budianto

Muhammad Budi Kusuma

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

INTRODUCTION

OBJECTIVES

1. To enable students to understand the basics of Java Programming
2. To enable students to understand Java Syntax

MODULE TARGETS

1. Students are able to write code in the Java programming language
2. Students are able to understand the syntax of the Java programming language

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop or personal computer
3. Intellij IDE/Any Java IDE (Eclipse /VSCode)
4. “Full Poweeeerrrrrrrr”

KEYWORDS

Java, Java Basic, if/else, Loop, Github

THEORY

1. What's Java?

1.1 Java As Programming Language

Java is a programming language born in the early '90s. To be exact, it was released in 1995 by James Gosling from Sun Microsystems. At that time, computers were growing super fast, but there was a problem: programs written on Windows couldn't run on Linux, programs on Linux couldn't run on Macintosh (Apple), and so on. Programmers had to write programs three times for each platform, and it was very inefficient. That's where the Java came in, with its principle: "Write Once, Run Anywhere."

Imagine Java is like a recipe. If you write a recipe in Bahasa Indonesia (for example, C++), only Indonesian people (Windows) will understand it. But when you write a recipe in Java, it gets compiled into a universal language (bytecode), so you can share your recipe anywhere you want. That way, every assistant chef (JVM) can read and follow the instructions to cook.

1.2 Compare Java With C++

Feature	C	Java
Core Philosophy	Full control & Performance ("Trust the programmer")	Safety & Portability ("Write Once, Run Anywhere")
Execution	**Compiled (Compile-only)**. Code is turned directly into machine language (e.g., .exe).	**Compiled & Interpreted**. Code is turned into **Bytecode** (.class), which is then executed by the **JVM**.
Portability	**Platform-Dependent**. Must be recompiled for each different OS.	**Platform-Independent**. The same Bytecode can run on any OS that has a JVM.
Memory Management	**Manual**. The programmer is fully responsible for using new and delete. Prone to *memory leaks*.	**Automatic**. Has a **Garbage Collector** (GC) that cleans up memory automatically.
Pointers	**Yes**. Provides direct, low-level memory access.	**No**. Uses *references* which are safer and managed (no *pointer arithmetic*).
OOP Paradigm	**Hybrid**. Supports OOP, but can also be used for purely procedural programming (you can have functions outside a *class*).	**Purer OOP**. Almost everything is an Object. Even main() must be inside a *class*.

Feature	C	Java
Performance	**Extremely Fast**. Because it's compiled to *native code* and has manual memory control.	**Fast**. Was originally slower due to the JVM, but modern *Just-In-Time (JIT) Compilers* make it very competitive.
Example Use Cases	Game engines (Unreal, Unity), Operating Systems, hardware drivers, high-frequency trading apps.	Enterprise applications (banking), web back-ends (server-side), Android apps, Big Data systems.
Structure Program	Should use a Header #Include <stdio.h> / #Include <stream.io> and program is should be on the int main method	Should use class before making main method, and the main method should be declared as public static method (Before the JDK 25)

2. Java Basic

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}

```

From those code we will learn Syntax of Java:

2.1 Class

Every Java code must be run in a Class, Class is a “blueprint” to create an object, in this context, the Main class is the foundation for our program, Class names should be written in PascalCase.

2.2 Method

To create a main method, you can write it as shown above. The word “main” should be written in lowercase. (Pro tip: you can just type “psvm” and then press Tab to auto generate the main Method).

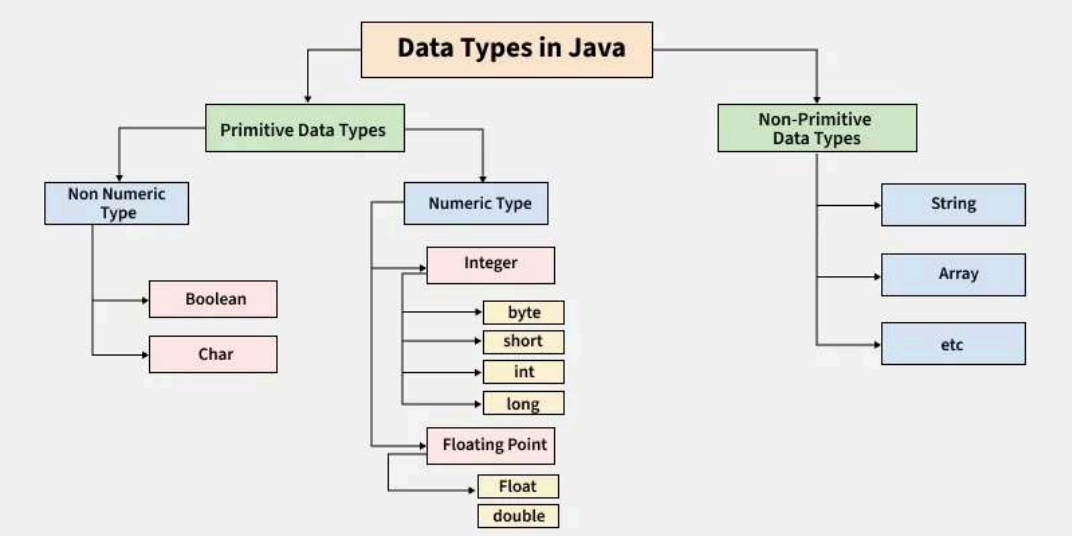
2.3 Output

The print method in Java is different from C or Python. In C, you write the print method as `printf("Hello, World!")` but in Java it’s a little bit longer, and there are 3 types of print methods in Java.

Print Method	Explanation	Shortcut	Output
`System.out.println( “Hello, World!”)`	Method print that included a newline.	sout	`Hello, World!` `Process finished with exit code 0`

Print Method	Explanation	Shortcut	Output
`System.out.print("Hello, World!")`	It's just a usual print with nothing.	soup	`Hello, World!` `Process finished with exit code 0`
`System.out.printf("Hello, World!")`	It's the usual print but with format, you can write print like in the C.	souf	`Hello, World!` `Process finished with exit code 0`

2.4 Data Types



Java is a statically typed programming language, which means variables are known at compile time. There are two defined data types: **Primitive** and **Non-Primitive**. Primitive data types store simple values directly in memory, while Non-Primitive (a.k.a. Reference) data types store memory references to objects. Primitive types depend on the data type for their size, while non-primitive types all have the same size.

Imagine that memory (RAM) is a desk. The desk (Stack) is for primitive data – it's easy to access. The drawer (Heap) is for non-primitive data – it stores bigger files. Primitive data is like writing your age on a sticky note and placing it on the desk. You have easy access to it, and it only saves your age (we'll talk about this later). If your friend sees your sticky note, writes down the same number, and places it on their desk, you'll have two sticky notes with your age. If you change your age on your sticky note, the age on your friend's desk will not change.

Non-primitive data is like drawers. Non-primitive data includes String, Array, Object, etc. For this analogy, you can't just write an object like "Mahasiswa" on your sticky note, so you write it in a file and save it in your drawer. As a replacement on your desk, you write the file location, like "drawer M-10". For non-primitive data, you save the memory address of your data, not the actual data. This means when you change the data in the drawer, the data will also change for your friend (since they have the same address pointing to the same drawer).

2.4.1 List of Primitive Data Types

Data Types	Category	Memory Size	Primary Use
byte	Integer	8-bit	Very small whole numbers. (e.g., age, game level)
short	Integer	16-bit	Small to medium whole numbers. (Rarely used)
int	Integer	32-bit	**Most common** for whole numbers. (e.g., counters, IDs)
long	Integer	64-bit	Very large whole numbers. (e.g., population data, transaction IDs)
float	Decimal	32-bit	Single-precision decimal numbers. (Must end with `f`)
double	Decimal	64-bit	**Most common** for decimals. (e.g., GPA, scientific calculations)
boolean	Logical	1-bit (logical)	Stores `true` or `false` values. (e.g., login status, if conditions)
char	Character	16-bit	Stores a **single** Unicode character. (Uses single quotes: `'A'`, `'%'`)

2.4.2 List of Non-Primitive Data Types

Data Types	Category	Primary Use
String	Object (Text)	Stores a sequence of characters (text). (Uses double quotes: "Hello")
Array	Object (Collection)	Stores a collection of data of the same type. (e.g., int[], String[])
Class	Object (Template)	A custom data type that you create. (e.g., Student, House, Scanner)
Interface	Reference	A "contract" that a Class can implement. (e.g., List, Map)
(Others)	Object	All other types that are not the 8 primitives. (e.g., Integer, Double - *Wrapper* Types)

3. Conditional Statement

Conditional Statement is a tool to make "if…else…", and it's a part of Control Flow, it's an ability to control the flow of a program. In here we will learn about how to control the flow of a program with if else and input.

3.1 Input Program

Input program in Java it's not the same as input in C, While in C we use "scanf", in Java we use Class called **Scanner,** it's a part of Java.Util Package within the Java Class Library. But in Java, it's not just Scanner, there are other classes that can handle an input like **Class Buffer** and **Class Console,** but in this module we will use Scanner Class. In order to use Scanner, we should import **Class Scanner** into our code first like this:

```
1 import java.util.Scanner;
```

After that, we should make an **object** from the **Scanner** with a name variable that we want in the main Method, for example we will declare Scanner object with a name as **input**:

```
1 import java.util.Scanner;
2
3 public class ModulePractice {
4   public static void main (String [] args){
5     Scanner input = new Scanner(System.in)
6   }
7 }
```

Let's break it down one-by-one. The code `public class ModulePractice` defines the name of our class (we'll talk about it in the next module). The code `public static void main (String [] args)` is the main method, every Java program will always start running from this main method. `Scanner input = new Scanner (System.in);` here, "Scanner" is a class that we imported earlier, and "input" is the name

of our Scanner variable, `new Scanner ()` means we create a new object from the Scanner class, and `System.in` means we get input from the keyboard while the program is running. This might be hard to understand just by reading it, so go ahead and practice it yourself.

Let's make a simple program to practice. Here, we will make a program that takes input for a name and age. Here's an example:

```
1 import java.util.Scanner;
2
3 public class ModulePractice {
4   public static void main (String [] args){
5     Scanner input = new Scanner(System.in)
6
7     String name;
8     int age;
9
10    System.out.print("insert your name: ");
11    name = input.nextline();
12    System.out.print("insert your age: ");
13    age = input.nextInt();
14  }
15 }
```

Let's break it down:

1. The variable `name` above is declared as a String.
2. The variable `age` is declared as an int.
3. `name = input.nextline();` is the code to input a String into the variable `name`. In this code, we use `input` as the name of our Scanner variable, then we call the method `nextline()` which is used to handle user input as a String that contains space.
4. `age = input.nextInt();` is the code for handling user input as an Integer into the variable `age`, the method `nextInt()` is only used for the int data type.

What about the other data types? Actually there are methods for other data types, here's the list:

1. `next()` : used to input a String but only until the next Space.

2. `nextDoubel()` : used to input the Double data type.
3. `nextFloat()` : used to input the Float data type.
4. `nextByte()` : used to input the Byte data type.
5. `nextBoolean()` : used to input the Boolean data type.

And there are many more, and you can check them right here.

Let's continue our program. As you can see above, we've already talked about output. Let's add output to our program so you can see your name and your age:

```java
import java.util.Scanner;

public class ModulePractice {
  public static void main (String [] args){
    Scanner input = new Scanner(System.in)

    String name;
    int age;

    System.out.print("insert your name: ");
    name = input.nextline();
    System.out.print("insert your age: ");
    age = input.nextInt();

    System.out.println(name);
    System.out.println(age);
  }
}
```

And here's the output:

```
insert your name: Yossua Agung
insert your age: 21
Yossua Agung
21

Process finished with exit code 0
```

We can add more to the output. For example, we can add a text "Name: " before displaying the user's name, so it looks more organized and user-friendly.

```java
import java.util.Scanner;

public class ModulePractice {
  public static void main (String [] args){
    Scanner input = new Scanner(System.in)

    String name;
    int age;

    System.out.print("insert your name: ");
    name = input.nextline();
    System.out.print("insert your age: ");
    age = input.nextInt();

    System.out.println("Your name: " + name);
    System.out.println("Your age: " + age);
  }
}
```

And the output will be like this:

```
insert your name: Yossua
insert your age: 20
Your name: Yossua
Your age: 20

Process finished with exit code 0
```

3.2 Condition

Conditions in Java are used to decide whether a block of code should run or not. This is based on the boolean value (true or false) from an expression.

❖ Switch/Case

This type of condition is another form of if/else if/else, the difference is that in this condition, we use keyword `switch` and `case`. The syntax of switch/case is the same as in C, here's an example:

```java
public class ModulePractice {
  public static void main (String [] args){
    String warna = "Merah";

    switch(warna){
      case "Merah":
        System.out.println("Warna Merah");
        break;
      case "Hijau":
        System.out.println("Benda itu Hijauu!");
        break;
    }
  }
}
```

And here's the output:

```
Warna Merah

Process finished with exit code 0
```

This condition works the same way as in C/C++ and is very similar to Java. Here's an example:

```c
#include <stdio.h>

int main(){
  char[5] warna = "Merah";

  switch(warna){
    case "Merah":
      printf("warna merah, warna cinta");
      break;
    case "hijau":
      printf("Benda itu hijauu")
        break;
  }
}
```

❖ If Condition

This condition executes a block of code only if the expression is true. If the condition is false, nothing happens. Example:

```java
public class ModulePractice {
  public static void main (String [] args){
    int umur = 20;

    if (umur > 18){
      System.out.println("Anda orang Dewasa");
    }
  }
}
```

Here's the Output:

```
Anda orang Dewasa

Process finished with exit code 0
```

❖ If/Else Condition

This is the same as above, but if the condition is false, an alternative block of code executes instead. Example:

```java
public class ModulePractice {
    public static void main (String [] args){
        int umur = 20;

        if (umur > 18){
            System.out.println("Anda orang Dewasa");
        } else {
            System.out.println("Anda Masih Kecikk");
        }
    }
}
```

❖ If/else if/else Condition

While the if/else condition only has 2 options, the if/else if/else condition allows us to have more than 2 options. Example:

```java
public class ModulePractice {
    public static void main (String [] args){
        int umur = 20;

        if (umur > 18){
            System.out.println("Anda orang Dewasa");
        } else if (umur > 10) {
            System.out.println("Anda Masih Remaja");
        } else {
            System.out.println("Anda masih Kecikk");
        }
    }
}
```

These if conditions work the same way as in C/C++. Here's an example:

```c
#include <stdio.h>

int main(){
  int umur = 20;

  if (umur > 18){
    printf("Anda Sudah Dewasa");
  }
  else if (umur > 10){
    printf("Anda Sudah Remaja");
  }
  else{
    printf("Anda masih Kecil")
  }
}
```

❖ Nested If

We learned about nested if statements in Basic Programming. The concept remains the same in Object-Oriented Programming. Example:

```java
public class ModulePractice {
    public static void main (String [] args){
        int umur = 20;
        boolean punyaGaji = true;

        if (umur > 18){
            System.out.println("Anda orang Dewasa");
            if (punyaGaji){
                System.out.println("dan punya pekerjaan");
            } else {
                System.out.println("dan tidak punya pekerjaan");
            }
        } else if (umur > 10) {
            System.out.println("Anda Masih Remaja");
        } else{
            System.out.println("Anda masih Kecikk");
        }
    }
}
```

❖ Loop

Loops in Java allow us to execute code repeatedly while a condition is true. This is useful for automating repetitive tasks. Here are some types of loops in Java:

❖ For

The syntax of a for loop:

```java
for (int i = 0; i<=10;i++){
  //the code that will be executed
}
```

The variable `i` tracks the loop counter. As long as `i` is less than or equal to 10, the code executes. `i++` adds 1 to `i` each iteration. The loop body is enclosed in `{ }`. Example:

```java
public class ModulePractice {
    public static void main (String [] args){
        for(int i=1; i<=5; i++){
            System.out.println("Module Practice Line " + i);
        }
    }
}
```

Output:

```
Module Practice Line 1
Module Practice Line 2
Module Practice Line 3
Module Practice Line 4
Module Practice Line 5

Process finished with exit code 0
```

This one is also similar to C/C++. Here's an example:

```c
#include <stdio.h>

int main(){
  int i;
  for(i = 0; i < 5; i++){
    printf("Loop ke: %d", i);
  }
}
```

❖ For Each

This loop is used to iterate through the values of an array. An array is a variable that stores more than one value and has an index (we'll learn about this in Module 5). The for-each loop uses the `for` keyword. Example:

```
for (int item : dataArray){
  //block that will run each iteration
}
```

The variable `item` holds each value from the array. We can read it as: "For each `item` in `dataArray`, execute the loop body." Example:

```
public class ModulePractice {
    public static void main (String [] args){
        int angka[] = {10, 20, 30, 40, 50};

        for(int x : angka){
            System.out.print(x + " ");
        }
    }
}
```

Output:

```
10 20 30 40 50
Process finished with exit code 0
```

❖ While

The while loop can be translated as “selama”, This loop works similarly to a condition (if/else). It will run as long as the condition is true. The syntax of a while loop:

```
while(condition){
  //the code will be run
}
```

The condition can be a comparison or any boolean value that evaluates to true or false. This loop will stop when the condition becomes false. Example:

```
public class ModulePractice {
    public static void main (String [] args){
        int angka = 0;

        while (angka < 5){
            System.out.println("Angka saat ini: " + angka);
            angka++;
        }
    }
}
```

Output:

```
Angka saat ini: 0
Angka saat ini: 1
Angka saat ini: 2
Angka saat ini: 3
Angka saat ini: 4

Process finished with exit code 0
```

This is similar to C/C++. Here's an example:

```c
#include <stdio.h>

int main(){
  int i = 0;
  while (i < 5){
    printf("ini angka ke: %d", i);
  }
}
```

❖ do-While

This loop is the same as the while loop, but do-while executes the code block at least once before checking the condition. Here's the syntax:

```
do{
  //the code will executed
}while (conditions);
```

For example:

```java
public class ModulePractice {
    public static void main (String [] args){
        int angka = 0;

        do {
            System.out.println("Angka saat ini: " + angka);
            angka++;
        } while (angka < 5);
    }
}
```

Output:

```
Angka saat ini: 0
Angka saat ini: 1
Angka saat ini: 2
Angka saat ini: 3
Angka saat ini: 4

Process finished with exit code 0
```

C/C++ also has this, here's an example:

```c
#include <stdio.h>

int main(){
  int i = 0;

  do {
    printf("Ini Angka Ke: %d", i);
    i++;
  }while (i < 5)
}
```

4. Practice

Let's try to make a simple program with Java. Try to rewrite this code, and let me explain it line by line

```java
import java.util.Scanner; // Mengimpor library untuk membaca input

public class Main {
    void main(String[] args) {
        // 1. Persiapan Scanner
        // System.in artinya kita membaca input dari keyboard
        Scanner scanner = new Scanner(System.in);

        // Variabel penampung total nilai
        double totalNilai = 0;

        System.out.println("=== Program Analisa Nilai Siswa ===");

        // Meminta jumlah siswa (Untuk menentukan batas perulangan)
        System.out.print("Masukkan jumlah siswa: ");
        int jumlahSiswa = scanner.nextInt();
        scanner.nextLine();

        // 2. FOR LOOP
        // Loop akan berjalan dari i = 1 sampai i <= jumlahSiswa
        for (int i = 1; i <= jumlahSiswa; i++) {
            System.out.print("Masukkan nilai siswa ke-" + i + ": ");
            double nilai = scanner.nextDouble();
            scanner.nextLine();

            // Menambahkan nilai ke total (untuk rata-rata nanti)
            totalNilai += nilai;

            // 3. IF - ELSE
            // Logika pengecekan status kelulusan per siswa
            if (nilai >= 75) {
                System.out.println("    -> Status: LULUS");
            } else if (nilai >= 50) {
                System.out.println("    -> Status: REMEDIAL");
            } else {
                System.out.println("    -> Status: TIDAK LULUS");
            }

            // Garis pemisah agar rapi
            System.out.println("-------------------------------");
        }

        // Menghitung rata-rata (Pastikan tidak membagi dengan 0)
        if (jumlahSiswa > 0) {
            double rataRata = totalNilai / jumlahSiswa;
            System.out.println("Rata-rata nilai kelas: " + rataRata);
        } else {
            System.out.println("Tidak ada data siswa yang dimasukkan.");
        }

        // Menutup scanner untuk mencegah kebocoran memori (memory leak)
        scanner.close();
    }
}
```

It's a program that checks if a student passes or not and calculates the average grade from all students by taking input from the user. First it asks how many students need to have their grade entered, then continues to take input and shows the average. I'll explain it line by line so you can understand it well.

```
import java.util.Scanner;
```

This is where we import the Scanner library. If we use Scanner, we need to import it first (Tip: if you want to import multiple libraries from the JDK, you can write `import module java.base`)

```
public class Main {
    void main(String[] args) {
```

This is the class name and main method we use. Just as a reminder, we need to run the main method inside a class, and usually we name our class `Main`, but you can name it whatever you want, as long as the class name is the same as the file name.

The second line is the main method. It's similar to C/C++, but in C/C++ we use `int main()`, while in Java we use `void main()`, previously, we uses `public static void main(String [] Args)` , but after the JDK 25 update, we can now use `void main()` as the main method.

```
System.out.println("=== Program Analisa Nilai Siswa ===");
System.out.print("Masukkan jumlah siswa: ");
int jumlahSiswa = scanner.nextInt();
scanner.nextLine();
```

This is where the program asks how many students will be graded. As you can see there's a variable `jumlahSiswa` that stores the number of students, and `scanner.nextInt()` gets an integer value from the scanner. But as you can see, there's also a `scanner.nextLine()` there. Why do we need that? The purpose is to clear the buffer, because `.nextInt()` only takes the integer. When we press Enter, the program reads it as a newline(`\n`), which is not an integer, so we need to clear the leftover buffer.

```
for (int i = 1; i <= jumlahSiswa; i++) {
    System.out.print("Masukkan nilai siswa ke-" + i + ": ");
    double nilai = scanner.nextDouble();
    scanner.nextLine();
```

This is where we use a loop to take the grades of every student. The grade value stored in the variable `nilai` . Because `nilai` is a double, we use `scanner.nextDouble()`, and then there's also a `scanner.nextLine()` to clear the buffer.

```
        totalNilai += nilai;
        if (nilai >= 75) {
            System.out.println("    -> Status: LULUS");
        } else if (nilai >= 50) {
            System.out.println("    -> Status: REMEDIAL");
        } else {
            System.out.println("    -> Status: TIDAK LULUS");
        }
```

We've already declared `totalNilai` before, and now we add `nilai` to `totalNilai`, so we can calculate the average later.

Now we have an if condition to determine whether the student passes, needs remedial, or fails based on the grade.

```
        if (jumlahSiswa > 0) {
            double rataRata = totalNilai / jumlahSiswa;
            System.out.println("Rata-rata nilai kelas: " + rataRata);
        } else {
            System.out.println("Tidak ada data siswa yang dimasukkan.");
        }
```

Here we have an if-else to check whether `jumlahSiswa` is not zero. This is called error handling or error prevention. If it's zero, then the program would divide by zero, which would cause the program to crash.

```
        scanner.close();
```

Last but not least, we close the scanner. Why do we need to close our scanner? It's preventing resource leaking.

5. Git & Github

Git is software based on a **Version Control System (VCS)** used to track changes to all files or projects that we create. Developers use this as a Distributed Version Control System (DVCS).

Github is a cloud service used to store and manage projects called **Repositories. Github** is fully online, so there's no need to install additional software.

Imagine a code project is like **playing a game**. In a game, if you want to reach another level, there are **checkpoints**. If you fail while trying to reach the next level, you don't have to restart from the very beginning, you respawn from the last checkpoint.

GitHub works the same way**.** A repository is like a checkpoint. If you write code that works well or has no errors, you push it to GitHub. If you want to add a feature to your code, but the feature doesn't work well or even breaks everything, you don't need to rewrite your code from scratch. You just need to go back to the last version you pushed to your repository, this is called pulling from the repository.

Note on terminology, **Push:** Sending your code changes to GitHub. **Pull:** Getting the latest code from GitHub.

Tips

Java Basic
Github For Beginner

Exercise

1. Rewrite this code

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Welcome to Labit RPG!");

        System.out.printf("Insert your name: ");
        String name = scanner.nextLine();

        System.out.printf("Insert your age: ");
        int age = scanner.nextInt();

        if(age < 13) {
            System.out.println("Sorry, you must be at least 13 years old to play this game.");
            scanner.close();
            return;
        } else {
            System.out.printf("Hello %s, age %d! Let's start your adventure!\n", name, age);
        }

        System.out.println("Let's Make some Charracter!");

        boolean continueCreation = true;
        do {
            System.out.println("Choose your class:");
            System.out.println("1. Warrior");
            System.out.println("2. Mage");
            System.out.println("3. Archer");
```

```
System.out.printf("Enter the number of your choice: ");
            int classChoice = scanner.nextInt();
            scanner.nextLine();

            switch (classChoice) {
                case 1:
                    continueCreation = false;
                    System.out.println("You have chosen the Warrior class! Strong and brave!");
                    // Additional Warrior setup can go here
                    break;
                case 2:
                    continueCreation = false;
                    System.out.println("You have chosen the Mage class! Wise and powerful!");
                    // Additional Mage setup can go here
                    break;
                case 3:
                    continueCreation = false;
                    System.out.println("You have chosen the Archer class! Agile and precise!");
                    // Additional Archer setup can go here
                    break;
                default:
                    System.out.println("Invalid choice. Please restart the game and choose a valid class.");
                    break;
            }
        } while(continueCreation);

        System.out.println("Character creation complete! Get ready for your adventure!");

    }
}
```

2. That code only includes name and age, you should also include home region.
3. Here's what you need to understand:

    a. Why should we name our class Main? Can we change it to something else?
    b. What's Scanner used for?
    c. What will happen if we input an alphabet instead of a number for age ?
    d. What happens when we input "10" for age? What does it mean?
    e. What is "do" on line 26 (first page of the code)?

Assignment

The mayor wants us to create a data collection system for basic building information. The mayor wants the program to follow these requirements:

The program must have these three features:

1. Add new Building (must take input from the user)
   The user will input the **name of the building**, the **address of the building**, and the **number of floors**. After that, the program will display the details of the building.
2. View All Buildings (there's nothing to do for now)
   There is nothing to do for now.
   Just add a dummy Message like "Coming Soon".
3. Exit
   Exits the program.

The expected output will look like this:

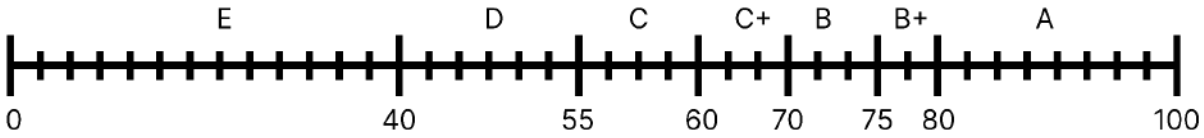
```
Welcome to Smart City Management System
1. Add New Building
2. View All Buildings
3. Exit
Please select an option: 1
Enter Building Name: Graha Merdeka
Enter Building Address: Jl. Sudirman No. 45
Enter Number of Floors: 20

================================
Building Name: Graha Merdeka
Building Address: Jl. Sudirman No. 45
Number of Floors: 20
================================
Building added successfully!
```

Assessment

ASSESSMENT ASPECT	POINT
Exercise	**0%**
Assignment	**100%**
Code Running Smoothly	30%
Originality of the code	10%
Code Explanation	60%
Total	**100%**

Percentage Assessment



A = (81 - 100) → You're The Chosen One
B+ = (75 - 80) → Great!
B = (70 - 74) → Good!
67 = (67) → Square root of 4489
C+ = (60 - 69) → Keep it up!
C = (55 - 59) → Almost there!
D = (41 - 54) → Study Some More!
E = (0 - 40) →

Closing Statement?

How was your day? Feeling good?
Brain still working? 😄
Is the material understandable?

If our material is great, let us know!
If you don't understand something… ASK THE ASSISTANT BROOOOO, ASKKKKKK!!
Seriously, don't be shy.

Have a great day. Here we learn Java basics.
Java is a very very very popular language.
If you want to become a backend developer one day, Java is one of the most used languages in banking companies.

Alright, that's it. Byeeeeeee